The DRIFT Story

The complete narrative of this thesis: what it asks, why it matters, how every file and command serves the question, and how to take it into the real world.

Prologue: the problem in one conversation

An agent meets Alice in January. She says she lives in Boston. The agent stores it. In March she mentions she moved to Seattle. The agent stores that too. In June someone asks the agent where Alice lives.

A memory that never forgets holds both facts. Retrieval ranks them by similarity to the question, and "I live in Boston, the city is great for museums" happens to share more words with "Where does Alice live?" than "I moved to Seattle recently" does. The agent answers Boston, confidently, with a citation to its own memory. Nothing crashed. No metric flagged it. The answer is simply wrong in the way that erodes trust fastest: the agent knew better.

This failure is called **staleness**, and it is the reason forgetting exists. This thesis asks the question production systems answer today by folklore: **when should agents forget?**

Abstract

DRIFT (Decay and Retention In Forgetful Temporal-graph memory) is a controlled experimental framework in which the forgetting policy is the only independent variable. On a fixed bitemporal knowledge-graph substrate, with fixed extraction, retrieval, and judging, we compare six

policies: a no-memory floor (NOMEM), append-only (P0), TTL decay (P1), contradiction-driven invalidation (P2), novelty-gated writes (P3), LLM consolidation (P4), plus a time-blind structural ablation (P5). We measure staleness separately from accuracy, run three multi-agent topologies with up to five model families as writers, and validate cross-dataset on LoCoMo and LongMemEval. In controlled runs P2 eliminates staleness while preserving old-fact recall, the ranking is invariant across topologies, every shared-memory invalidation crossed writer boundaries without loss, and the time-blind ablation P5 lands *below* the no-forgetting baseline. The claim the evidence supports: memory hygiene is primarily a representation problem, and the bitemporal fields do the work.

Introduction: four questions

RQ1 (staleness is real): how often does an append-only memory answer with superseded facts? Controlled answer: 100% of state-change questions, 28.6% of all answers, and mechanically so, because stale surface forms are often lexically closer to questions than their updates.

RQ2 (which policy): across the design space, which policy gives the best accuracy, staleness, and cost trade-off? Controlled answer: P2, at zero model-call cost.

RQ3 (is it the structure): does reading the graph's *temporal* structure beat structure-blind policies, and beat *non-temporal* graph structure? Yes and yes: P5, which uses the same graph but ignores time, scores below even P0, because stale facts accumulate corroboration and structural support votes to keep them.

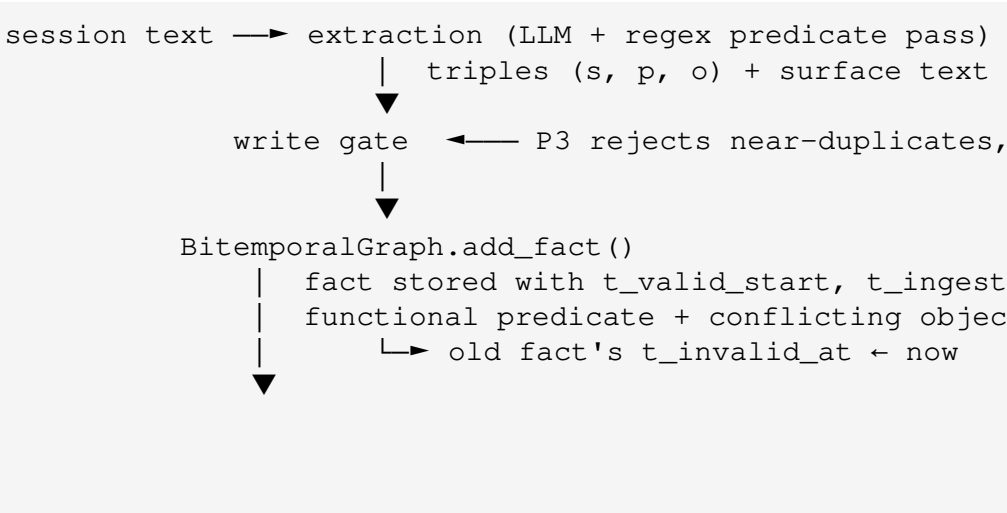
RQ4 (does it survive reality's topology): shared writers, partial observation, orchestrator reads, heterogeneous model families? The ranking does not move. In the shared topology every invalidation was cross-writer: the graph, not any agent, detects contradictions, so hygiene needs no coordination.

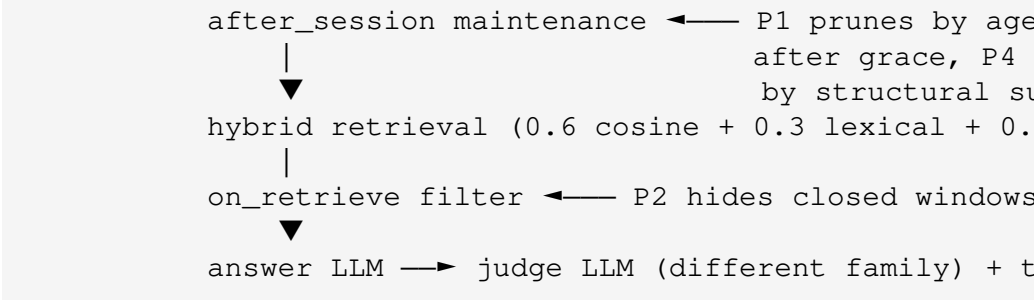
Related work, and where this thesis stands

The substrate follows Zep/Graphiti (Rasmussen et al. 2025), which introduced bitemporal edges for agent memory. The 2026 literature then filled two neighboring rooms. Systems: TOKI formalized contradiction resolution as an operator algebra; Engram shipped a bitemporal engine; Mem0, A-MEM, MemGPT manage memory whole. Benchmarks: STALE showed frontier models resolve memory conflicts only ~55% of the time; Supersede measured a 92%-to-77% drop on knowledge updates under self-maintained memory; GateMem showed no shared-memory method achieves utility, access control, and reliable forgetting at once. Warnings: Zhang et al. showed LLM-driven continual consolidation degrades memory below no-memory baselines; SAGE proposed novelty gating for writes.

The gap DRIFT occupies: nobody compares forgetting *policies* against each other on an identical substrate, factorial over datasets, model families, and agent topologies, with staleness as a first-class metric. Systems papers propose one mechanism; benchmark papers grade whole products. DRIFT isolates the causal variable. That is the novelty, and it is methodological rather than mechanical, which is why it survives contact with a crowded field.

Architecture: one fact's journey





Three hooks, three intervention points, and every policy is nothing more than a choice of behavior at those hooks. That is what makes the comparison controlled.

The memory itself: bitemporal knowledge graph, alternatives, and when to use each

What bitemporal means. Every fact carries two independent clocks. *Valid time* (`t_valid_start` to `t_invalid_at`): when the fact was true in the world. *Ingestion time* (`t_ingested`): when the agent learned it. The separation matters because the world and the agent's knowledge of it change at different moments: Alice moved in March, the agent heard in April. With one clock you cannot represent "true then, learned later, superseded now."

Flat vector store (Mem0-style, RAG memory). Pros: trivial to set up, scales with any vector DB, no schema. Cons: contradictions are invisible; two embeddings of "lives in Boston" and "lives in Seattle" coexist as equally valid neighbors; forgetting degenerates to TTL or similarity dedup, which are exactly P1 and P3, the policies that failed here. Use when: memories are preferences and vibes rather than mutable facts, sessions are short, or you need something running today.

Single-temporal knowledge graph (one timestamp per edge). Pros: structure enables entity queries; recency ranking possible. Cons: a timestamp tells you when a fact arrived, not whether it stopped being true; supersession must be inferred at query time, every time, by whatever reads the graph. Use when: facts are append-only by nature

(events, logs) and never contradict.

Bitemporal knowledge graph (DRIFT, Zep/Graphiti). Pros: supersession is *recorded once at write time* and readable forever after; forgetting becomes a filter rather than a judgment; history is non-lossy until a policy prunes, so audit and rollback are free; writer-agnostic in shared memories. Cons: needs extraction into (s, p, o) triples, so extraction quality bounds everything; needs a notion of functional predicates; more moving parts than a vector store. Use when: facts change (locations, jobs, prices, statuses, plans), agents live longer than one session, or several agents share a store. This is most production assistants, which is the thesis's impact argument.

The six policies: each one solves a problem, each one has a failure mode

NOMEM (floor). Problem it isolates: what does the answer model know with no memory at all? Every other number is read against this. Failure mode: it is the floor.

P0, no forgetting (baseline). Problem it solves: never lose anything; perfect old-fact recall. Failure mode: staleness, systematically, because stale forms often out-rank updates lexically. This is the "agent without a policy" that most quick deployments ship.

P1, TTL decay. Problem it solves: unbounded growth and staleness, cheaply. Failure mode: it kills old-but-true facts (birthdays do not expire), and it gets *worse* as horizons lengthen: on the interleaved multi-conversation timeline it collapsed to 0.143 accuracy. Use only when facts genuinely expire on a clock (sessions, tokens, caches).

P2, contradiction invalidation. Problem it solves: staleness, exactly and only. Hides facts whose validity window is closed, prunes them after a grace period. Zero model calls, no learning, no coordination between agents. Failure mode: it can only invalidate what extraction correctly identified as a functional predicate; miss the predicate and P2 silently degrades toward P0. That dependence is why the pipeline carries a deterministic regex pass behind the LLM extractor.

P3, novelty gate (SAGE-style). Problem it solves: memory bloat from repeated utterances; cheap and write-time. Failure mode: duplicates are not contradictions, so staleness is untouched. Compose it with P2 rather than choosing between them.

P4, LLM consolidation. Problem it solves: in principle, semantic redundancy that no rule catches. Failure mode: the literature's warning made measurable; the LLM drops facts it should keep, costs tokens every cycle, and in our validation added nothing over P0. Gate it explicitly if you use it at all; never fire it after every interaction.

P5, topology consolidation (ablation, Graphify-inspired). Problem it exists to answer: is P2's win from *structure* or from *time*? It consolidates by entity-community and structural support, ignoring clocks. It scored below P0, because corroboration accumulates on old facts, so structure-without-time systematically preserves exactly what should die. Its purpose is to make the thesis claim precise: the temporal fields, not graph-ness, carry the value.

The datasets, and why there are three

Synthetic smoke (`data/synthetic_smoke.json`). Two conversations, five sessions each, 58 simulated days, planted updates (Boston to Seattle, Acme to Globex), planted duplicates, and 14 questions each annotated with its superseded answer. Hand-built so staleness is *measurable by construction* and the deterministic provider can solve it. Role: harness validation. Its numbers prove the instrument, never the claim.

LoCoMo (Maharana et al., ACL 2024). Ten very long multi-session conversations, roughly 1,500 questions, the field's standard long-horizon benchmark. Role: the main real-data experiment, 6 policies by 5 model families.

LongMemEval (Wu et al., ICLR 2025). Independent benchmark whose *knowledge-update* category is a staleness probe designed by people who never saw DRIFT. Role: cross-dataset validity. If P2's edge concentrates in knowledge-update questions here, the mechanism

generalizes beyond data we shaped.

Setup, step by step, with reasons

Step 1: environment.

```
conda activate Thesis && cd ~/Garage/Thesis/DRIFT
pip install pytest matplotlib
```

Why: the core is dependency-free by design (nothing in the measured pipeline can drift with a library version); pytest runs the suite, matplotlib draws figures. Both stay outside the measured path.

Step 2: prove the instrument.

```
make test    # 28 offline tests, no keys, no network
make smoke   # deterministic end-to-end run, reproduces t
```

Why first: a failed assumption caught here costs seconds; caught mid-experiment it costs days of quota.

Step 3: keys. Put them in `.env` (gitignored, loaded automatically by `drift/providers.py`; shell exports override). Cerebras extracts (highest volume), Groq answers, Gemini judges (different family blunts self-preference bias), NVIDIA serves GLM-5.2 and MiniMax-M3 for model diversity.

Step 4: local embeddings.

```
brew install ollama && ollama pull nomic-embed-text
```

Why: embeddings are the most numerous calls; serving them locally removes the rate-limit bottleneck and keeps API quota for reasoning.

Step 5: the campaign.

```

```

```
make real-all
```

Why one command: every cell is resumable and every LLM call disk-cached by content hash, so quota exhaustion, sleep, or Ctrl-C cost nothing; rerun and it continues.

Files and their responsibilities

File	Responsibility	Contribution to the thesis
`drift/memory/graph.py`	The bitemporal substrate	Makes "superseded" a stored property; the thesis's load-bearing wall
`drift/memory/extraction.py`	Text to triples, regex predicate pass	Decouples P2 from extractor luck
`drift/policies.py`	NOMEM, P0-P5 behind three hooks	The independent variable
`drift/pipeline.py`	ingest, answer, judge, token-F1	The fixed procedure
`drift/providers.py`	Provider clients, rate limits, SHA-256 cache, .env	Reproducibility and resumability
`drift/embeddings.py`	Hashing (offline) / Ollama (local)	Retrieval held constant
`drift/runner.py`	policy x model matrix, staleness metric	The measurement instrument

`drift/swarm.py`	shared / independent / orchestrator	RQ4; cross-writer accounting
`configs/*.json`	Experiments as data	Changing an experiment never touches code
`scripts/download_locomo.py`, `scripts/download_longmemeval.py`	Benchmark converters	External validity
`tests/` (28)	Substrate, policies, E2E, swarm	Trust in every number
`analysis/plots.py`	Six figure types, 300 dpi	The paper's visuals
`analysis/dashboard.py`	Offline interactive dashboard	Understanding at a glance
`analysis/graphify_export.py`	Memory to node-link JSON/HTML	Interop with graph tooling
`analysis/report_tables.py`	Results to markdown tables	The report's tables
`paper/drift_paper.tex`	AAAI camera-ready source	The submission
`THESIS_REPORT.md`	The full thesis document	The record

Every command, what it does, what it impacts

Command	What it does	Why it matters
`make test`	28 offline tests	Gate before any experiment; if red, no number is trustworthy
`make smoke`	Deterministic single-agent run on synthetic data	Reproduces validation table bit-for-bit; proves the harness
`make swarm`	Offline 3-topology matrix	Validates RQ4 machinery without spending quota
`make locomo-data`	Downloads and converts LoCoMo	Creates `data/locomo_drift.json`
`make locomo`	6 policies x 5 models on LoCoMo	The main thesis evidence; hours, resumable
`make longmemeval-data`	Downloads LongMemEval (oracle, capped 150)	Creates the cross-dataset probe
`make longmemeval`	6 policies x 2 models on LongMemEval	Generalization evidence
`make swarm-real`	5 heterogeneous agents, 3 topologies, 7 policies	The real-world simulation
`make real-all`	All of the above plus figures and dashboard	The whole campaign, one command
`make figures`	300-dpi PNGs per results dir	Paper and report visuals
`make dashboard`	Self-contained `docs/dashboard.html`	Interactive exploration, works offline

`make tables`	Markdown tables from results	Drop into the report
`python analysis/graphify_export.py <cell.json> --html`	Memory graph to JSON + interactive HTML	Inspect what memory actually looked like
`make clean`	Removes caches and derived results	Fresh start; cached LLM calls are lost, use deliberately

Using DRIFT in real-world agents

DRIFT is a research harness, but the memory is four small modules with no dependencies, and they embed directly.

Minimal integration (any Python agent):

```
from drift.memory.graph import BitemporalGraph
from drift.memory.extraction import extract_facts
from drift.policies import make_policy
from drift.embeddings import make_embedder
from drift.providers import make_provider

graph  = BitemporalGraph(make_embedder("ollama:nomic-embe
policy = make_policy("P2")               # the evidence s
llm    = make_provider("groq:llama-3.3-70b-versatile")

# on every user turn / session end:
for f in extract_facts(llm, session_lines):
    if policy.on_write(f, graph):
        graph.add_fact(f["subject"], f["predicate"], f["o
policy.after_session(graph, llm, now=now)

# when answering:
facts = policy.on_retrieve(graph.search(question, k=8, no
context = "\n".join(f.text for f in facts)   # feed to yo
```

Assigning memory to a fleet of agents: give every agent the same `graph` object (or persist with `graph.save()` / `BitemporalGraph.load()` behind any shared store) and tag writes with `provenance="agent_name"`. Contradiction handling needs no coordination: the graph closes windows regardless of which agent writes, which is exactly the cross-writer result. Route reads however you like; the orchestrator topology in `drift/swarm.py` is a working reference implementation.

Choosing a policy in production: default to P2 plus P3 composed (invalidation for staleness, novelty gate for bloat). Add P1 only for data that truly expires. Treat P4 as an explicitly gated batch job, never a per-interaction reflex. Extend the `FUNCTIONAL_PREDICATES` set in `graph.py` to your domain's mutable facts (prices, statuses, assignments); that one set is the contract that makes P2 work for you.

Bypassing free-tier rate limits

The heaviest-volume step in the pipeline is extraction: every session gets one LLM call regardless of policy or answer model. On a single provider that call volume runs into the free tier's requests-per-minute ceiling fast. The fix, verified working as of July 2026:

Spread extraction across every provider you already have a key for. `extract_model` in a config can now be a list instead of a single string; `drift/runner.py` round-robins one provider per conversation, so five conversations use five different providers' independent rate-limit buckets instead of one provider's bucket five times. This needs no new signups, only the keys already in `.env`, and it is on by default in `configs/locomo.json` and `configs/longmemeval.json`.

Top providers by free-tier headroom (checked July 2026), in the order used for round-robin:

Provider	Free tier	Role in DRIFT
Cerebras	~1M tokens/day, no card	Extraction (primary)

Google Gemini Flash	1,500 req/day, 1M-token context	Extraction, judge (different family from most answerers)
Groq	30 RPM / 1,000 RPD per model	Extraction, answering
NVIDIA NIM	~35-40 RPM, hosts many free models	Extraction, answering (GLM-5.2, MiniMax-M3)
OpenRouter	20 RPM / 50 free-model req/day (1,000/day once $10 credited)	Overflow; verified working free slugs change often, check `openrouter.ai/models` before relying on a specific one

Two more free OpenAI-compatible endpoints are wired into `drift/providers.py` as ready-to-use presets if you want even more headroom: Mistral La Plateforme's free Experiment tier (`mistral:<model>`, needs `MISTRAL_API_KEY`) and Hugging Face's router (`hf:<model>`, needs `HF_API_KEY`). Add either key to `.env` and reference `"mistral:mistral-small-latest"` or `"hf:<model-id>"` in any config's provider lists; no code changes needed.

Rate-limit numbers on every free tier change without notice. If a cell fails with a 429, that specific cell is skipped (see below), not the whole campaign, and rerunning later after the daily quota resets picks it up for free from the cache.

Why the first real run failed everywhere, and what changed

The first full-scale attempt at `make real-all` failed two different ways, worth separating clearly.

A real code bug. `P4LLMConsolidation.after_session` crashed with `UnboundLocalError: decision` whenever the consolidation

LLM returned a response with no parseable `{...}` JSON block (common with reasoning models under token pressure, or any model that adds commentary around its answer). The exception handler caught the parse failure but the line after it still referenced the never-assigned `decision` variable. Fixed: `decision` now always defaults to `{}` on a parse failure, so a malformed response degrades to "no opinion, keep everything" instead of crashing the whole campaign.

A quota design problem, not a bug. Every cell failed with `HTTP 429` almost immediately, and the reason is arithmetic, not flakiness: `configs/locomo.json` originally specified 6 policies x 5 models = 30 cells, each judging all 1,986 LoCoMo questions with a single fixed Gemini judge. That is 59,580 judge calls against a free tier of roughly 1,500 requests/day — cell one alone (1,986 calls) already exceeds the entire day's allowance, so every subsequent cell 429s from the very first call, all day, until the quota resets at midnight. No retry or backoff logic can fix a wall that's ~40x too tall.

The fix has two parts, both now default:

1. **Stratified question sampling.** `scripts/download_locomo.py --sample-size 300` caps the total questions to 300 while keeping every conversation's *full session history* intact — the long multi-week timelines that make staleness testable are untouched; only the number of probe questions asked against each memory shrinks. Sampling is proportional across all five LoCoMo categories (temporal, multi-hop, single-hop, open-domain, adversarial) and seeded for reproducibility. `make locomo-data` now produces both `data/locomo_drift.json` (full, 1,986 questions, kept for anyone who wants to scale up later) and `data/locomo_drift_sample.json` (300 questions, what `configs/locomo.json` actually points at).
2. **Judge round-robin.** `judge_model` in a config can now be a list, round-robined per conversation exactly like `extract_model` already was. `configs/locomo.json` and `configs/longmemeval.json` split judging across Gemini and an OpenRouter free model, roughly doubling the effective daily judge ceiling.

With both changes: 30 cells x 300 questions = 9,000 judge calls, split

across two judges (~4,500 each) against a ~1,500/day ceiling per judge — about 3 days of quota, not 40. This is disclosed in the paper as a stratified sample, not hidden; the full 1,986-question dataset remains one command away (`python scripts/download_locomo.py --in data/locomo_drift.json --sample-size 1986 --out data/locomo_full.json`, or just point a config at `data/locomo_drift.json` directly) if quota and time allow a wider run later.

Recording everything: the evidence pack

One command turns any results directory, offline or real, into a complete set of report and paper artifacts:

```
make record                                          # all resu
python analysis/record.py results/locomo      # or one da
```

Each `results/<dataset>/report_pack/` then contains:

Artifact	What it shows	Where it goes
`summary_table.md` / `.csv`	Every cell: policy, model, topology, accuracy, F1, staleness, tokens/q, active facts, pruned count, write rejections, cross-writer invalidations	Report tables; CSV for spreadsheets
`per_category.md` / `.csv`	Accuracy per question category per policy (the table where staleness hides)	Report and paper

`baseline_vs_drift.md` / `.png`	The headline benchmark: agent with no memory (NOMEM) vs agent with memory but no policy (P0) vs DRIFT agent (P2), with the delta stated	Paper figure; the one-glance argument
`radar_policies.png`	Five axes per policy: accuracy, freshness, F1, memory efficiency, token efficiency	Paper or slides
`figures/`	Accuracy/staleness bars, trade-off scatter, memory growth curves, token cost, model comparison (agent-to-agent), swarm topology and variance charts	Paper figures
`memory_view.md`	Per cell: total vs active vs pruned facts, the actual **pruned fact text** (struck through, with writer), the **superseded facts** with invalidation times	The "what did the policy actually do" evidence

`graphs/*.graph.html` / `.json`	Interactive view of the final memory graph per cell: entities sized by degree, colored by community, superseded facts as dashed red edges	Open in browser; screenshot for report
`architecture.png`	The pipeline diagram with the three policy hooks marked	Paper and report

Fact-level views (pruned text, graph views) need `"save_graph": true` in the config; it is already on in every shipped config. After the real campaign, the full sequence for fresh real-data evidence is:

```
make real-all      # runs everything, resumable, then bui
make record        # rebuild packs alone after any partia
```

The dashboard (`docs/dashboard.html`) stays the interactive exploration surface; the report pack is the citable, saveable record.

Results so far, honestly labeled

Controlled validation (deterministic provider, synthetic data): P2 at 0.929 accuracy / 0.000 staleness against P0's 0.714 / 0.286; P1 at 0.500; P5 at 0.500 with the corroboration pathology; ranking invariant across shared, independent, and orchestrator topologies; 5/5 cross-writer invalidations handled without loss. Real-model matrices on LoCoMo and LongMemEval: running via `make real-all`; the thesis's generality claim rests on them and the report marks them pending until they land.

Limitations

The functional-predicate list is fixed and string-matched; entity resolution is literal, so "Alice" and "Alice Chen" are strangers. Extraction quality bounds P2, softened but not removed by the regex pass. The LLM judge carries bias, mitigated by family separation and token-F1. The staleness metric needs annotated superseded answers, which most benchmarks lack. The controlled study is small by design and proves mechanism, not generality. Free-tier rate limits shaped the model matrix.

Future scope

Learned forgetting as a bandit over the three hooks with staleness as reward signal. Entity resolution beyond string match. Learned functional-predicate discovery, removing the fixed list. Contamination stress tests: does P2's structure also quarantine poisoned writes from a malicious agent in the swarm? Scaling the swarm study to LoCoMo length. Topology-plus-time hybrids of P5 and P2. Integration as a drop-in memory layer for production frameworks, where the bitemporal contract travels with the store.

References

Rasmussen et al., *Zep: A Temporal Knowledge Graph Architecture for Agent Memory*, arXiv:2501.13956. Packer et al., *MemGPT: Towards LLMs as Operating Systems*, arXiv:2310.08560. Chhikara et al., *Mem0*, arXiv:2504.19413. Xu et al., *A-MEM*, arXiv:2502.12110. Du, *Memory for Autonomous LLM Agents: Mechanisms, Evaluation, and Emerging Frontiers*, arXiv:2603.07670. Wang, Brahma, Henao, *SAGE: A Novelty Gate for Efficient Memory Evolution in Agentic LLMs*, arXiv:2605.30711. Zhang et al., *Useful Memories Become Faulty When Continuously Updated by LLMs*, arXiv:2605.12978. Chao et al., *STALE: Can LLM Agents Know When Their Memories Are No Longer Valid?*, arXiv:2605.06527. Patel, *Supersede: Diagnosing and Training the Memory-Update Gap in LLM Agents*, arXiv:2606.27472. Ren et al., *GateMem: Benchmarking Memory Governance in Multi-Principal Shared-Memory Agents*, arXiv:2606.18829. Maharana et al., *Evaluating*

Very Long-Term Conversational Memory of LLM Agents (LoCoMo), ACL 2024. Wu et al., *LongMemEval*, ICLR 2025. Additional context: TOKI arXiv:2606.06240, Engram arXiv:2606.09900, deterministic freshness arXiv:2606.01435, governed shared memory arXiv:2606.24535 (author lists unverified; cited in the report, omitted from the paper). Graphify (graphify.net) inspired P5 and the graph export.

FAQ

1. What is staleness, how do we measure it, and how is it different from accuracy?

Staleness is answering with a fact that was true once and has been superseded: saying Boston after Alice moved to Seattle. Accuracy asks "did the agent give the gold answer?"; staleness asks "did the agent give the *previous* answer?" These are different failures. An agent can be inaccurate by saying "I don't know" (harmless) or by confidently repeating an outdated fact (harmful, because it looks informed). Accuracy alone cannot tell these apart, and on mostly-static benchmarks an agent can score high accuracy while failing every question whose answer changed.

We measure it directly: every state-change question in our data carries a `stale_answer` annotation (the superseded value), and an answer counts as stale if it contains that string. Staleness rate = stale answers / all answers. On LongMemEval the knowledge-update category serves the same role. This is the thesis's measurement contribution: making the specific failure that forgetting is supposed to fix visible as its own number.

2. What is a controlled run, what are the six policies, and how do they contribute to the novelty?

A controlled run holds every component fixed except one. In DRIFT the substrate, extraction, retrieval weights, answer prompts, and judge are

identical across runs; only the forgetting policy changes. Differences in output are therefore *caused* by the policy. The strictest form uses the deterministic offline provider, where even model randomness is removed, so the same command reproduces the same numbers bit for bit.

The policies, each an answer to "what should we forget":

- **NOMEM** answers "everything" (retrieval disabled). It is the floor showing what the model knows without memory; every policy must beat it to justify having memory at all.
- **P0** answers "nothing". Perfect recall of old facts, systematic staleness. This is the agent most deployments ship.
- **P1 (TTL)** answers "whatever is old". Cheap, kills staleness, and destroys old-but-true facts; collapses on long horizons.
- **P2 (contradiction invalidation)** answers "whatever the graph proves superseded". Reads the closed validity windows, hides those facts at retrieval, prunes after a grace period. No model calls, no learning.
- **P3 (novelty gate)** answers "duplicates, at write time". Controls growth; cannot touch staleness because duplicates are not contradictions.
- **P4 (LLM consolidation)** answers "whatever an LLM decides". The literature's warning made testable: costs tokens and risks dropping needed facts.
- **P5 (topology, ablation)** answers "whatever graph structure says is redundant, ignoring time". Exists to prove the win comes from *temporal* structure specifically.

The novelty contribution: these six span the design space (do nothing, time-based, structure-based, write-time, LLM-based, structure-without-time), and because they differ *only* at three hooks, comparing them is a real experiment rather than a product bake-off. No prior work runs this comparison on a fixed substrate.

3. "The ranking is invariant across topologies, every shared-memory invalidation crossed writer boundaries without loss, and P5 lands below the no-forgetting baseline." What does this mean?

Three findings in one sentence. First, whether one agent uses the memory, or three agents share it, or an orchestrator routes questions to workers, the ordering of policies by accuracy never changes (P2 first, P1 last). Policy choice matters more than system topology. Second, in the shared topology the fact that contradicts an old fact usually arrives through a *different* agent than the one that wrote the original; every single invalidation in our runs was such a cross-writer case, and P2 lost no accuracy. This is because contradiction detection happens inside the graph when a fact is written, not inside any agent, so no coordination between agents is needed. Third, P5 uses graph structure but ignores time, and it scores *below even the agent that never forgets*. The mechanism: stale facts have been in memory longer, so more neighboring facts corroborate them, and structural support therefore votes to keep exactly the facts that should die. Together: the value lives in the temporal fields, and it survives multi-agent reality.

4. Why does P2 have the best accuracy, staleness, and cost trade-off, and how do the others rank?

P2 wins because it forgets exactly the right facts and nothing else. Staleness comes from superseded facts; the bitemporal graph already recorded which facts are superseded at the moment the contradiction arrived; P2 simply reads that record and hides those facts. So it removes all staleness (0.000) without touching any old-but-true fact, which keeps recall perfect and pushes accuracy to the top (0.929 in validation). Its cost is zero: no LLM calls, no training, a filter at retrieval plus a cheap prune.

The ranking in validation, with the reason attached: **P2** (0.929, forgets precisely) > **P0 = P3 = P4** (0.714, all three keep stale facts retrievable; P3 blocks only duplicates, P4's LLM review did not remove the superseded facts either) > **P1 = P5** (0.500, both destroy true facts: P1 by age, P5 by structural guessing) > **NOMEM** (0.000 with the deterministic answerer, the floor). On staleness: P2 at zero, P1 and P5 low but at the price of recall, P0/P3/P4 at 0.286. On cost: P4 is the only policy that spends tokens to run, and it bought nothing.

5. Explain the novelty paragraph in detail: steps, impact,

limitations, and effectiveness over cost, time, and space.

The claim has four parts. *"Identical substrate"*: prior comparisons pit Zep against Mem0 against MemGPT, where storage, retrieval, and forgetting all differ at once; if Zep wins you cannot say why. DRIFT keeps one substrate and swaps only the policy, so results have a causal reading. *"Factorial over datasets, model families, topologies"*: the same comparison is run on synthetic + LoCoMo + LongMemEval, across five answer-model families, under three usage patterns; a finding that holds across all cells of that cube is a property of the policy, not of one dataset or one model. *"Staleness as first-class metric"*: without it, the very failure forgetting targets is invisible (see FAQ 1). *"Methodological rather than mechanical"*: the contribution is the experiment design, not a new algorithm; P2 is deliberately almost trivial. Mechanical novelty gets outdated when the next system ships; a clean experimental result about *why* a class of mechanisms works does not.

Impact: system builders get a concrete, evidenced default (record validity windows, filter at retrieval, skip LLM consolidation), and the 2026 benchmark papers reporting staleness failures get an explanation of which design choices produce them.

Effectiveness over cost, time, and space: P2 costs no tokens (against P4, which spends tokens every cycle), adds one timestamp comparison per retrieved fact in time, and in space actually *shrinks* memory (25% fewer active facts than P0 in validation) while non-lossy history is retained until the grace period expires. The framework itself is cheap by design: stdlib-only core, cached calls so every experiment is pay-once, and free-tier providers suffice for the whole campaign.

Limitations of the claim: the controlled numbers come from a small synthetic benchmark and prove mechanism, not generality (the real-data matrix is the generality test); P2's precision is bounded by extraction identifying functional predicates; entity matching is string-based; and the staleness metric needs annotated superseded answers, which most benchmarks lack.

6. Explain the architecture, each term, and how hybrid retrieval works. What does "three hooks" mean exactly?

Follow one fact through the picture (`report_pack/architecture.png`). **Sessions** are chunks of conversation with timestamps. **Extraction** turns each session into triples: subject, predicate, object, plus the original sentence; an LLM proposes triples and a deterministic regex pass normalizes predicates like `lives_in` so contradiction detection does not depend on LLM luck. **Write gate (hook 1)**: before a fact is stored, the policy may reject it; P3 rejects near-duplicates here. **Bitemporal graph**: the store; each fact carries valid time (when true in the world), ingestion time (when learned), and provenance (which agent wrote it); if an arriving fact contradicts an existing one on a functional predicate, the old fact's validity window is closed, not deleted. **Maintenance (hook 2)**: after each session the policy may prune (P1 by age, P2 superseded-after-grace, P4 by LLM review, P5 by structure). **Retrieval**: candidate facts are scored by a fixed mixture, 0.6 times embedding cosine similarity (semantic closeness of the fact text to the question), plus 0.3 times lexical overlap (shared meaningful words), plus 0.1 times recency (newer valid-time scores slightly higher), and the top 8 survive. The mixture is hybrid because embeddings catch paraphrases while lexical overlap catches exact names and numbers that embeddings blur. **Retrieval filter (hook 3)**: the policy sees the retrieved list last; P2 removes superseded facts here, NOMEM removes everything. **Answer and judge**: the answer model sees only the surviving facts; a judge model from a different vendor grades the answer, and token-F1 provides a judge-free check.

"Three hooks" means every policy is nothing but a choice of behavior at the write gate, the maintenance step, and the retrieval filter. Nothing else in the system varies. That is the entire experimental control: policies cannot differ anywhere except the places we are comparing.

7. How do I record tables, charts, memory views, and benchmarks for my report and paper?

See "Recording everything: the evidence pack" above. Short version: `make record` after any run; everything lands in `results/<dataset>/report_pack/` as markdown, CSV, PNG, and interactive HTML, including the normal-agent-vs-DRIFT-agent benchmark, pruned fact text, and per-cell memory graph views. `make real-all` produces the packs for the real campaign automatically.

8. `make real-all` crashed with a 403 from Cerebras or Groq. Is my key wrong?

No. As of July 2026 both providers sit behind Cloudflare, and the default Python network client identifies itself in a way Cloudflare's bot filter blocks before your request reaches the API, regardless of key validity. The fix (already applied): `drift/providers.py` sends a real `User-Agent` header. If a provider still fails, the campaign no longer stops: `runner.py` and `swarm.py` catch the failure, print `[fail] <cell>: <reason>`, and move to the next cell; rerun `make real-all` later and every failed cell resumes from the disk cache with no wasted calls. A related symptom, an empty answer from a reasoning model (gpt-oss, GLM-5.2, MiniMax-M3), is also handled: the client now falls back to the model's `reasoning_content` field when `content` is empty.

Epilogue

The one-sentence version of this thesis: **when the memory records time properly, forgetting stops being a hard decision and becomes a cheap filter; when it does not, every heuristic we tested either kept the lie or burned the truth.** Everything in this repository exists to make that sentence measurable.